

How my AI coding team works

One smart AI becomes a whole team: a boss who thinks, helpers who do the busy work, a critic who hunts for mistakes, and safety guards that stop big accidents.

The motto: **cheap hands, expensive eyes.**

Let fast, cheap helpers do the running around. Keep the smartest brain for thinking, judging, and checking.

First, what problem does this solve?

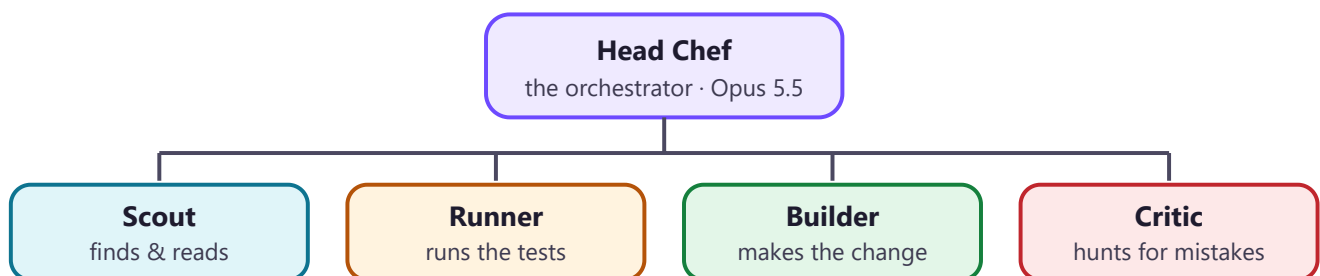
Claude Code is an AI that can read and write computer code. If you let **one** AI do everything by itself, two things go wrong:

- **It's expensive.** The smartest AI model costs the most. Using it to read hundreds of files is like hiring a top surgeon to sweep the hospital floor.
- **Its memory fills up.** An AI can only hold so much text in its head at once. This is called its **context**, and it's measured in **tokens** (small pieces of words). When it gets too full, the AI gets slower and starts forgetting details.

The fix: run it like a restaurant kitchen

A great restaurant doesn't have the head chef chop every onion. The head chef plans the menu, gives clear orders, and **tastes every dish before it leaves the kitchen**. The kitchen helpers do the chopping, stirring, and timing.

The Coding Orchestrator is a set of **rules, helpers, and safety tools** that makes Claude work the same way. The "orchestrator" is the head chef (like the conductor of an orchestra, who doesn't play every instrument but keeps everyone together).



Meet the team

Each team member is an AI with one job. They use different AI **models** (different "brains"): bigger brains think better but cost more, so each job gets the brain it actually needs.

HEAD CHEF · THE ORCHESTRATOR · OPUS 5.5 (THE BIG THINKER)

Plans, decides, and checks everything

Talks to you, figures out what you really need, splits the work into small jobs, and writes each helper a clear set of instructions (a **brief**). It makes the hard calls: design choices, finding the real cause of a bug, anything about security. And it **checks the helpers' work** before telling you it's done.

SCOUT · SONNET (FAST & CHEAP)

The researcher

Finds files, reads code, and sends back a short summary. It only **reads**; it never changes anything. This saves the head chef from stuffing its memory with whole files.

RUNNER · SONNET (FAST & CHEAP)

The tester

Runs the exact command it's given (like "run all the tests") and reports what happened: did it pass or fail, and the exact error. It doesn't try to fix anything. Test output can be thousands of lines long; the runner boils it down.

BUILDER · SONNET (FAST & CHEAP)

The maker

Makes a change that has **already been decided** ("add this button in this file"), then runs a check to prove it works. The thinking about *what* to build was done by the head chef. The builder never commits or pushes.

CRITIC · FABLE (SHARP-EYED REVIEWER)

The fault-finder

Its only job is to find what's **wrong**. It starts with "fresh eyes": it doesn't see the head chef's reasoning, so it can't be talked into agreeing. Called in for risky changes: security, deleting data, things other programs depend on.

Small jobs skip the team. If something takes about three steps or fewer, the head chef just does it. Writing a brief for a helper would take longer than doing the job. (But a risky small job still gets the critic.)

Who does what, and with which brain

The golden rule about big actions

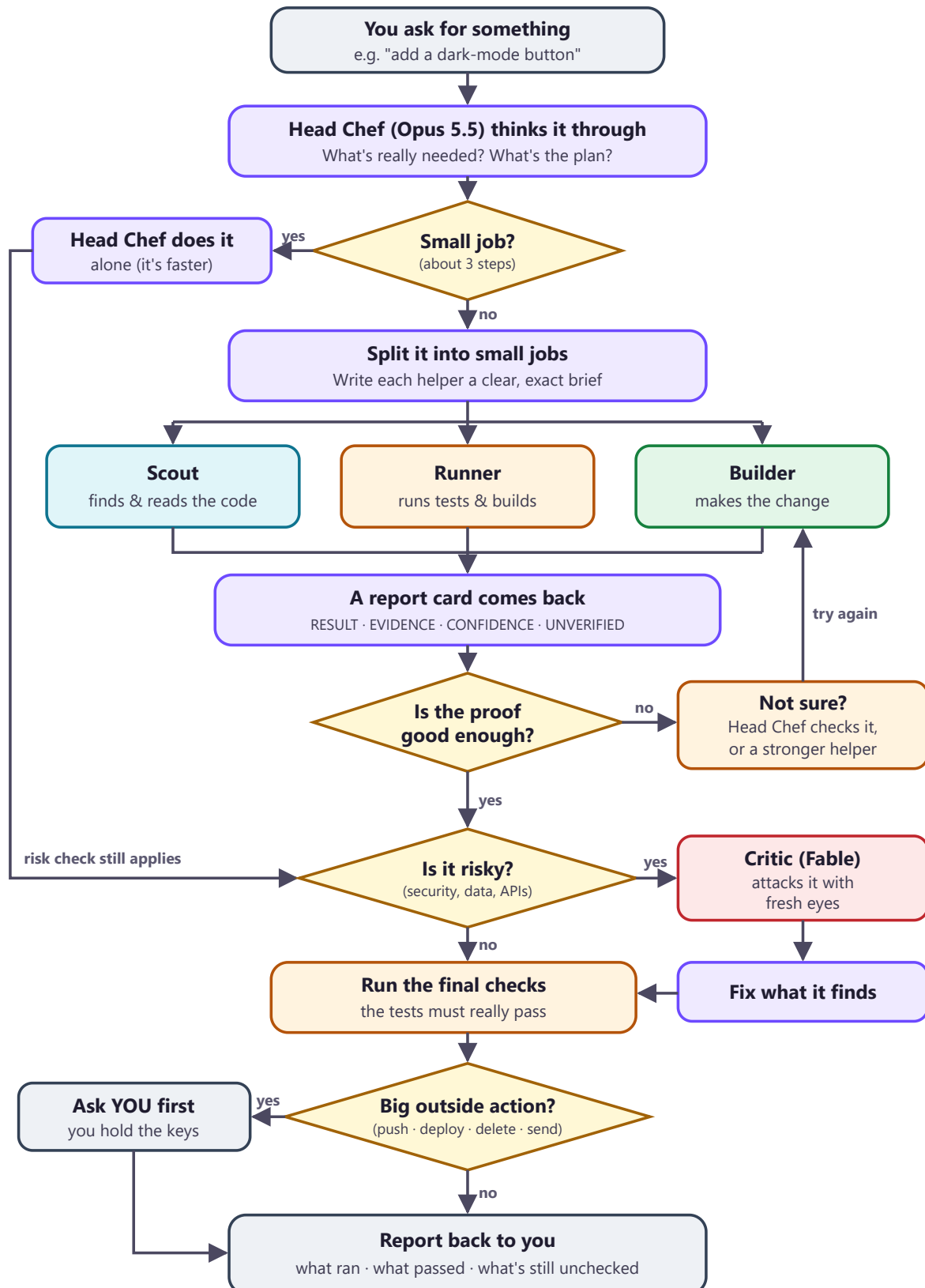
Some actions reach outside your computer and are hard to undo: **pushing** code to GitHub, **publishing, deploying** a website, **deleting** things, and **sending** messages. Helpers are never allowed to do these. Only the head chef can, and only when **you** have said yes.

Which model does what

Role	Claude Code	Codex	Why that one
Head chef	Opus 5.5	GPT-6.1 Sol · medium	Hard decisions and final checking need the strongest brain.
Scout	Sonnet	GPT-6 Luna · low	Reading and summarizing: fast and cheap is enough.
Runner	Sonnet	GPT-6 Luna · low	Just runs a command and reports what came out.
Builder	Sonnet	GPT-6.1 Sol · medium	Writes code from a plan that's already decided.
Critic	Fable	GPT-6 Astra · high	A different, sharp brain that's good at spotting mistakes.
Jev	jev-latest (TypeSafe)		A small, fast judge for quick questions and scores. It can move a helper to a bigger brain (Sonnet → Opus/Fable) when a job is hard.

Flowchart 1: the life of a request

Here's what happens from the moment you ask for something until you get an answer. Diamonds are questions; follow the "yes" or "no" arrow.



Purple = head chef · blue/orange/green = helpers · red = critic · yellow diamonds = questions

A real example, step by step

Say you ask: "Add a dark-mode button to my app."

1. **Head chef thinks.** Where do colors live in this app? Is there already a settings screen? This is too big for three steps, so it splits the work.
2. **Scout goes looking.** Brief: "Find where the app's colors and settings screen are defined. Don't change anything." Scout comes back with: "Colors are in `theme.dart` lines 10–40; settings screen is `settings.dart`."
3. **Head chef designs the change** using that summary, without having to read every file itself.
4. **Builder makes it.** Brief: "Add a dark theme in `theme.dart` and a switch in `settings.dart`. Then run the settings tests." Builder returns its report card *and* the actual change (called a **diff**), which the head chef reads.
5. **Runner runs all the tests** and reports: "312 passed, 0 failed, exit code 0."
6. **Risky?** A color switch isn't a security change, so no critic needed this time.
7. **Head chef reports to you:** what changed, which tests ran and passed, and anything it couldn't check (for example, "I didn't look at it on a real phone").

The report card

Every helper must answer with the same four sections. It's like showing your work in math class: an answer without working gets marked down.

RESULT

What happened, in one or two lines.
"Added the dark-mode switch."

EVIDENCE

Proof: commands run and their real results.
"Tests: 14 passed, exit code 0."

CONFIDENCE

How sure: high, medium, or low.
"High."

UNVERIFIED

What it did not check.
"Didn't test on iPhone."

Why not just trust the helpers?

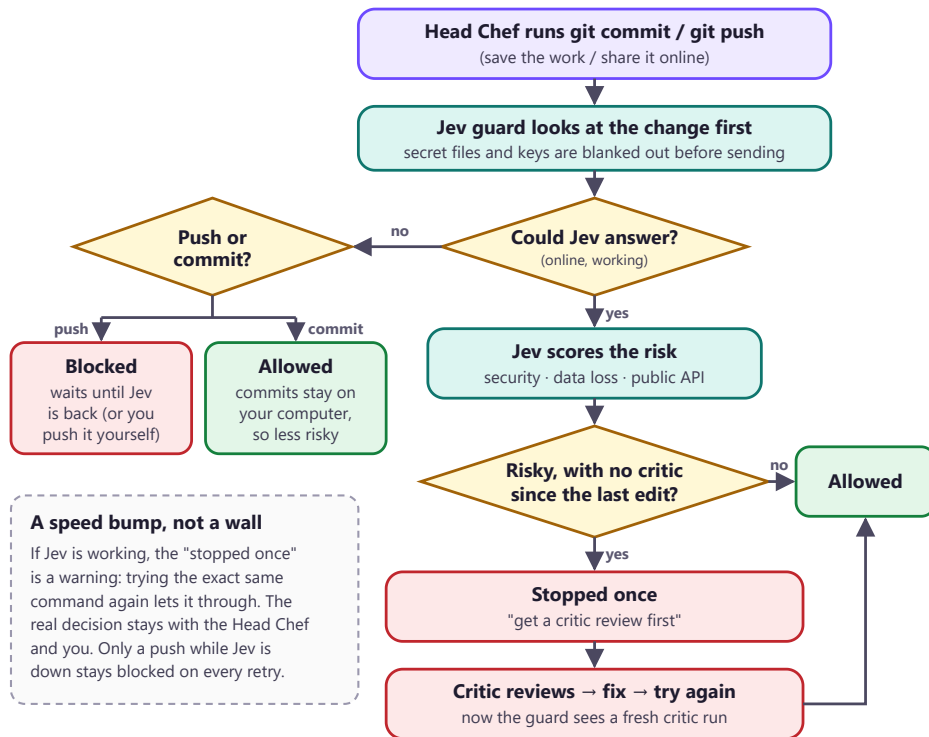
Helpers use cheaper, faster brains, so they sometimes make mistakes or sound more sure than they should. The rules say:

- If the evidence is **missing**, or confidence is only **low or medium**, the head chef doesn't just ask the same helper again. It **checks it itself**, or **moves the job up one level** (scout or runner → builder → head chef).
- The head chef always **reads the builder's actual change**, not just the summary.
- Checks must be run **after the last edit**. A test that passed before a change proves nothing about after it.
- "I think it works" is not a result. A result is a command that ran, plus its **exit code** (0 means success; anything else means something failed).

Fun fact: the critic is handed only the change and the claim to attack, never the head chef's conclusions. If you tell a reviewer "I'm pretty sure this is fine," they tend to agree. Fresh eyes catch more.

Flowchart 2: the safety guard (Jev)

Jev is a small, very fast AI judge from a company called TypeSafe. It's an optional add-on (switched on here). It never writes code. It just answers quick questions like "how risky is this?" in a split second. The orchestrator uses it in a few places. The most important one is a **guard at the door** before code is saved into history (`git commit`) or shared online (`git push`).



Teal = Jev · green = allowed · red = stopped

Jev's other jobs

Job	What Jev does
Pick the brain size	Before a helper starts, Jev reads the task and suggests which model should do it. It moves a job to a bigger brain easily (doing a hard job with a small brain is the expensive mistake) but needs to be very sure before moving one to a smaller brain. The critic is never moved.
Check report cards	If a helper's report is missing a section, or the evidence doesn't back up the result, it's sent back once to fix it.
Spot a topic change	If your new message is about something totally different, it says "start a fresh session" (see the next page).
Grade handoff notes	Scores the "baton" note from 0 to 4 (next page). A weak note isn't saved until it's fixed.
Answer questions about files	<code>ask-jev</code> lets the head chef ask "which of these files handles logins?" without reading them all into its own memory.

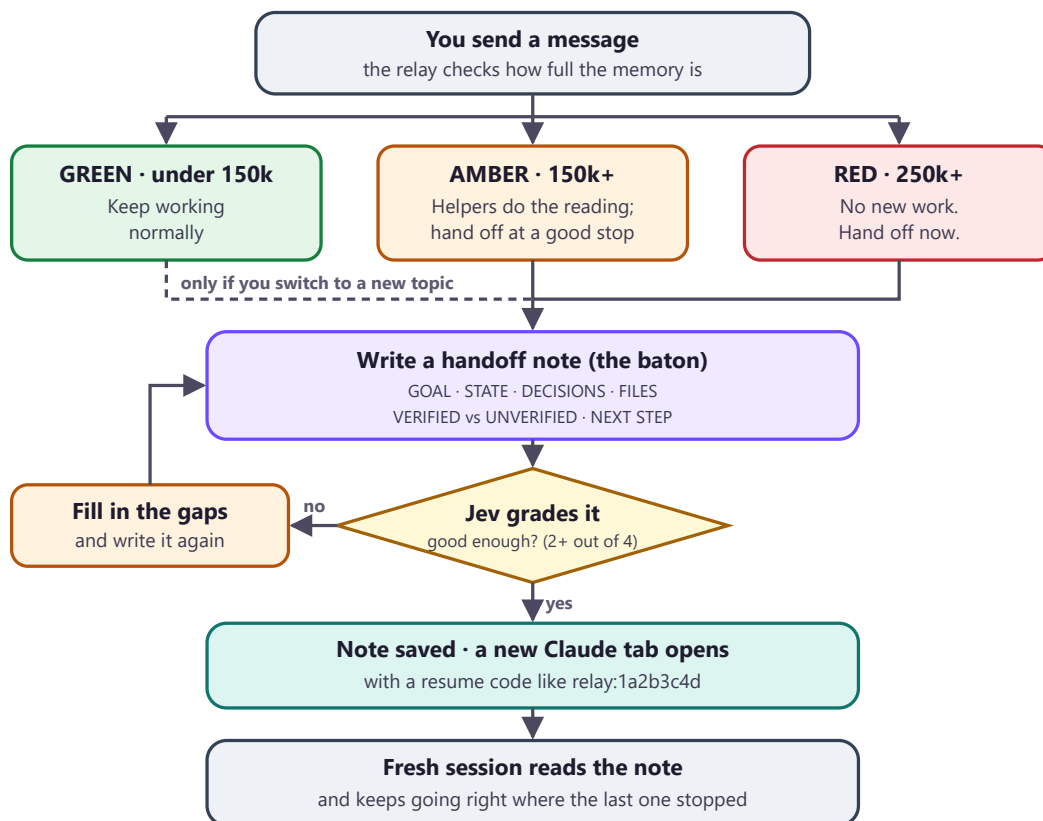
If Jev is offline, almost everything just works the way it did without Jev ("fails open"). The one exception is pushing code online, which is too important to wave through unchecked ("fails closed").

Flowchart 3: the relay race (when memory fills up)

Remember the AI's memory (its **context**) has a limit. A little program called the **relay** checks how full it is every time you send a message, and shows a fuel gauge:



Like a relay race: when one runner gets tired, they pass the **baton** to a fresh runner, who carries on from exactly the same spot. Here the baton is a written **handoff note**.



What goes in the handoff note?

GOAL	What we're trying to achieve.
STATE	What's done and what's left.
DECISIONS	Choices made and why, plus ideas that were rejected (so the next session doesn't try them again).
VERIFIED vs UNVERIFIED	What was actually tested (with the command and result) versus what's only assumed. The new session re-checks the unverified parts.
NEXT STEP	The exact next thing to do.

This very document was made after a handoff like this! The previous session filled up, wrote a note, and a fresh session picked it up.

The big rules, in one page

1. **The boss thinks; the helpers fetch.** Design, tricky bugs, and security decisions stay with the head chef. Reading, running tests, and following clear instructions go to cheaper helpers.
2. **Small jobs: just do them.** If it's about three steps or fewer, no helpers.
3. **Clear orders.** Every brief says the goal, the exact files, the rules, how to check it worked, and what to send back.
4. **Show your work.** Every report has RESULT, EVIDENCE, CONFIDENCE, and UNVERIFIED, with real exit codes.
5. **Don't trust, verify.** Weak proof means the head chef checks it, or the job moves up a level. Never just "try the same thing again."
6. **Risky means critic.** Security, deleted data, and changes other programs depend on all get a fresh-eyes review before they count as done.
7. **You hold the keys.** Push, publish, deploy, delete, and send: only the head chef does these, and only with your permission.
8. **Pass the baton.** When memory runs low, write a great handoff note and continue in a fresh session.
9. **Tell the truth.** The final report says what ran, what passed, and what wasn't checked. Failures are reported word for word.

Words to know

Orchestrator	The "boss" AI that plans and coordinates, like a conductor or head chef.
Subagent	A helper AI the orchestrator starts for one job (scout, runner, builder, critic).
Model	The AI "brain" being used. Opus 5.5 is the big thinker; Sonnet is fast and cheap; Fable is the critic's reviewer.
Context / tokens	The AI's short-term memory, and the small word pieces it's measured in.
Brief	The instructions given to a helper.
Diff	The exact list of lines that were added or removed in a change.
Exit code	The number a program gives when it finishes: 0 = success, anything else = a problem.
git commit / push	Commit = save a snapshot of the code on your computer. Push = upload those snapshots to GitHub for others.
Hook	A little program that runs automatically at certain moments (like before a command runs). The relay and the Jev guard are hooks.
Jev	A fast AI judge (from TypeSafe) that answers quick "how risky / which one / is this good enough" questions.
Handoff / relay	The note and the system that let a fresh session continue the work.